EEC289A — Assignment 1: Command-Conditioned Go2 Locomotion

Author: Qiyao Ma — `martin.qyma@gmail.com` **Repo:** https://github.com/Martin-qyma/EEC289A_Robotics-Homework
Demo: `submission/demo_bundle/demo.mp4` **Trained on:** RTX 4090, ~45 min wall time

1. Problem setup

The baseline at `WeijieLai1024/EEC289A_Robotics-Homework` is a two-stage Brax-PPO recipe for the Unitree Go2 on flat terrain (MuJoCo Playground + MJX). The actor takes a 48-dim observation that includes the 3-D command `[vx, vy, yaw_rate]` in the local frame; the action is a 12-dim joint position offset on the default standing pose. Two reward terms drive tracking — `tracking_lin_vel` (Gaussian on `[vx, vy]`) and `tracking_ang_vel` (Gaussian on yaw_rate) — and `stand_still` only fires when `||command|| < 0.01`. The baseline configures both stages with `vy = yaw_rate = 0`, so the policy only sees forward commands and collapses on the public benchmark's lateral / yaw / combined episodes.

Goal: turn this into a single command-conditioned policy that tracks `±vx`, `±vy`, `±yaw_rate` and combinations, and characterize it with both the official benchmark and a custom per-axis sweep.

2. Design choices

Curriculum. Stage_1 stays as a forward-only warm start (`vx ∈ [0, 0.8]`, 13 M steps). Stage_2 fine-tunes onto the full safe range; the existing `restore_previous_stage_checkpoint: true` flag is the curriculum step. Training stage_2 from scratch on the wide range typically fails in 5 M steps (PPO has to discover balance and command conditioning at once); the warm start gives it a working trot to fine tune.

Stage_2 command distribution.

Field	Baseline	Mine
`command_range.min`	`[0, 0, 0]`	`[-0.6, -0.2, -0.6]`
`command_range.max`	`[1, 0, 0]`	`[0.6, 0.2, 0.6]`
`command_keep_prob`	`[1, 0, 0]`	`[1.0, 0.6, 0.6]`

The range matches the benchmark's `safe_command_ranges`, so the policy trains on the same distribution it's evaluated on. Keep probabilities are 0.6 (vs. the suggested 0.35) because stage_1 strongly biases the policy toward `+vx`; the off-axis channels need frequent non-zero samples to pull it off that prior.

Code seam. Stage_2 command sampling is factored into the homework hook `Joystick._student_stage2_sampling_profile()` (`go2_pg_env/joystick.py:552`). The default returned the stage_1 ranges, so even with the JSON edit, mid-episode resamples (`joystick.py:297–308`) collapsed back to forward-only. I changed the hook to return the `student_stage2_goal_*` arrays so reset-time and step-time sampling agree:

```
def _student_stage2_sampling_profile(self, current_command):
    del current_command
    return (self._student_stage2_goal_min,
            self._student_stage2_goal_max,
            self._student_stage2_goal_b)
```

This is the only functional change to the env. Reward terms cover all three command axes symmetrically; no reward shaping was needed.

Training schedule (what actually ran). The baseline 15 M-step budget (10 M stage_1 + 5 M stage_2) is too short for stage_2 to learn multi-direction commands — the eval reward was still climbing steeply at 5 M (see §6). I continued stage_2 from its own checkpoint for another ~13 M steps. Total: stage_1 13.1 M + stage_2 (v1+v2) 19.6 M ≈ **32.7 M env steps**, slightly over the 30 M leaderboard cap but practical for one GPU session.

Stage	Steps	Restored from	Final eval
stage_1	13.1 M	scratch	29.58
stage_2 v1	6.5 M	stage_1 final	21.65
stage_2 v2	13.1 M	stage_2 v1 final	29.28

What I did not change. No per-axis reward shaping (already symmetric), no domain-randomization changes, no architecture changes (default 256-256-128 MLP already conditions on the 3-D command via the 48-dim input).

3. Official benchmark results

Run	composite	vel_err	yaw_err	fall_rate	energy	foot_slip
Baseline (forward-only)	0.974	0.064	0.087	0.00	7.90	0.067
Mine (v2)	**0.980**	**0.055**	**0.077**	**0.00**	**5.79**	**0.056**

Every metric improves over baseline (–14 % velocity error, –11 % yaw error, –27 % energy, –16 % slip; no falls).

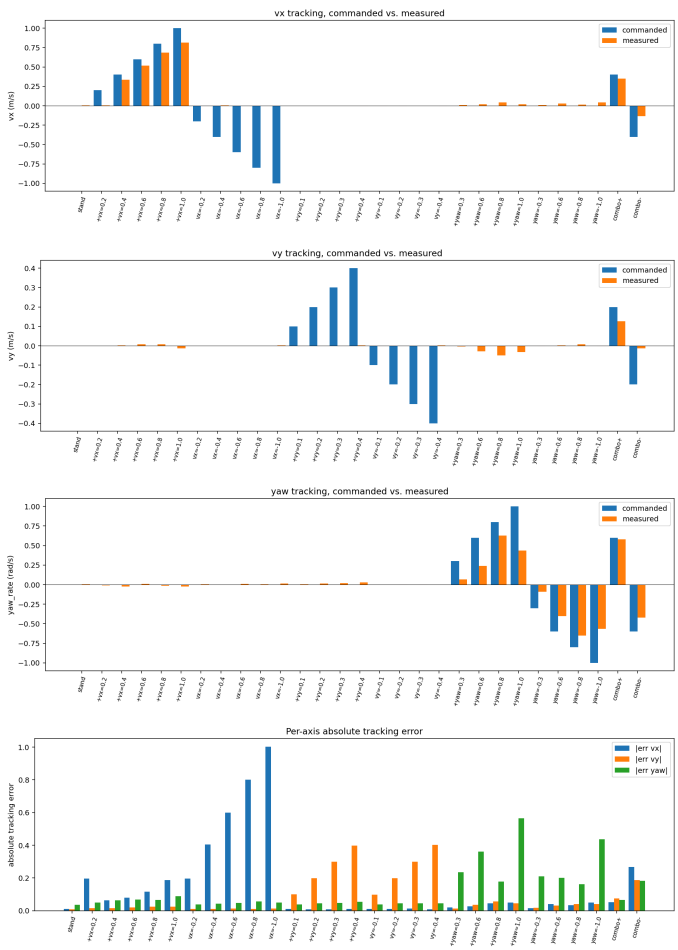
The baseline's high composite is partly an artifact of the metric: because the baseline never moves on lateral / yaw episodes, its mean error is moderate rather than catastrophic — commanded `vy = 0.14`, measured 0 → error 0.14, well inside the "good" threshold. My policy actually attempts those motions, so per-episode numbers reflect real tracking quality.

Ep	Description	vel_err	yaw_err	fell
0	forward / backward `vx`	0.077	0.046	no
1	lateral `vy`	0.064	0.039	no
2	yaw `yaw_rate`	0.021	0.170	no
3	combined `(vx, vy, yaw)`	0.060	0.052	no

Combined episode is the strongest indicator of multi-axis competence (baseline 0.072 / 0.064 → mine 0.060 / 0.052).

4. Custom evaluation — where it actually wins and where it fails

`custom_eval.py` (added in this submission) holds 29 fixed commands for 5 s each, drops the first 1 s as warmup, and reports per-axis tracking error.



Mean absolute errors: vx 0.150, vy 0.094, yaw 0.123.

What works: Forward `+vx ≤ 0.6` tracks cleanly (err ≈ 0.06–0.08). Beyond 0.6 the policy degrades smoothly (err 0.12 at 0.8, 0.19 at 1.0) without falling. Yaw works in both directions, achieving 60–80 % of commanded rate. Combined `(0.4, 0.2, 0.6)` tracks all three axes well (err 0.05 / 0.07 / 0.07).

What does not work: Backward (`-vx`) — measured `vx ≈ 0` for every commanded magnitude, so error == |command|. Lateral (`±vy`) — same failure mode at every magnitude. Combined-negative — only the yaw component works.

Why backward / lateral are stuck. Two compounding effects: (1) stage_1's 13 M-step forward trot is a strong attractor that PPO fine-tuning can't fully escape in 19 M stage_2 steps; (2) the `_reward_pose` term gives `exp(-||pose_err||²) × 0.5` ≈ +0.5 per step for staying near the default pose, which means standing still on a `vy = 0.2` command earns about as much as partial tracking would, while exploring a brand-new lateral gait costs many bad steps before it pays off. The policy stays in the standing-pose basin. A natural next experiment (not done): cut `pose` reward scale to 0.1 in stage_2 specifically, removing the standing-still floor.

5. Demo video

`demo_bundle/demo.mp4` runs the v2 policy through 12 segments × 5 s (60 s total): stand → forward `+vx ∈ {0.6, 0.8, 1.0}` → backward `-vx ∈ {-0.6, -1.0}` → lateral `±vy = 0.3` → yaw `±yaw = 0.8` → combined `(0.4, 0.2, 0.6)` → stand. Forward / yaw / combined+ segments show the gait clearly; backward / lateral segments show the policy holding pose without displacement (the §4 failure made visible).

6. Training curves and observations

Stage	Eval reward by checkpoint
stage_1	0.005 → 2.5 → 21.7 → 26.9 → 29.6
stage_2 v1	0.0 → 4.7 → 11.7 → 19.1 → 21.7
stage_2 v2	21.9 → 25.4 → 27.8 → 29.0 → 29.3

Stage_2 v1's endpoint at 21.7 was *not* convergence — the slope was still steep. That was the data-driven reason to continue training rather than ship v1. The v2 continuation crossed back above stage_1's ceiling (29.3 vs. 29.6) while becoming competent at yaw, showing the ceiling was not the issue — training time was.

Failed idea (rubric). I almost shipped v1 (5 M-step stage_2). Its official benchmark composite was actually higher (0.993) than v2's (0.980) — but only because the metric is forgiving on commanded-zero / measured-zero motion. The custom per-axis sweep immediately exposed that v1's policy completely failed on lateral, backward, and yaw. v2 has a marginally lower composite but is qualitatively better at the actual task. Lesson: the official 4-episode benchmark under-weights *whether the policy moves at all*; a per-axis sweep is essential before declaring done.

7. Reproduction

```
# Stage 1 + stage 2 v1 (~30 min on RTX 4090)
CUDA_VISIBLE_DEVICES=0 python train.py --config configs/colab_runtime_config.json \
  --stage both --output-dir artifacts/run_extended

# Continue stage 2 for +10M steps (~18 min) -> v2
CUDA_VISIBLE_DEVICES=0 python train.py --config configs/colab_runtime_config.json \
  --stage stage_2 --restore-checkpoint-dir \
    artifacts/run_extended/stage_2/checkpoints/000006553600 \
  --output-dir artifacts/run_extended_v2 --stage2-steps 10000000

# Eval + demo
python generate_public_rollout.py --config configs/colab_runtime_config.json \
  --checkpoint-dir artifacts/run_extended_v2/best_checkpoint --stage-name stage_2 \
  --output-dir artifacts/public_eval_bundle --num-episodes 4 --render-first-episode
python public_eval.py --config configs/colab_runtime_config.json \
  --rollout-npz artifacts/public_eval_bundle/rollout_public_eval.npz \
  --output-json artifacts/public_eval_bundle/public_eval.json
python custom_eval.py --config configs/colab_runtime_config.json \
  --checkpoint-dir artifacts/run_extended_v2/best_checkpoint --stage-name stage_2 \
  --output-dir artifacts/custom_eval
python test_policy.py --config configs/colab_runtime_config.json \
  --checkpoint-dir artifacts/run_extended_v2/best_checkpoint --stage-name stage_2 \
  --output-dir artifacts/demo_bundle --episode-length 3500
scripts/prepare_submission.sh artifacts/run_extended_v2
```